Data Analytics & Business Intelligence Department Handbook

Training, Project Engineering and Client Delivery Operating System
Beginner to Professional · 0 to 100% Project Lifecycle

Developed by **V. LOKESHWARAN**
VEL Software Solutions
velsoftwaresolutions.in@gmail.com

Document VEL-DA-001 · Version 1.0 · Module 1: Foundation, Onboarding and 90-Day Plan

Contents

Part 0 — How to Use This Handbook 3

Part 1 — Department Charter and Commercial Services 6

Part 2 — Organisation Structure 8

Part 3 — Employee Onboarding Plan 10

Part 4 — Training Roadmap: Levels 1 to 7 12

Part 5 — The 90-Day Training Schedule 14

Part 6 — Technology Stack and Selection 16

Part 7 — The 17-Stage Client Project Lifecycle 18

Part 8 — Teaching Format and Repository Standard 20

Part 9 — Commercial Model and Pricing 22

Part 10 — Mandatory Training Projects 24

Part 11 — Official Learning Sources 25

Part 12 — First Practical Assignment (A1): Audit a Sales Dataset 27

Part 13 — Module Roadmap and Progress Tracker 32

Annex A — Data Audit Report Template (Stage 04 / Assignment A1) 33

Annex B — Daily Log Template 33

Annex C — Quiz Q1 (Level 1, Week 1) and Trainer Answer Key 33

All prices and market figures in this handbook's examples are placeholders until verified from official sources (Part 9 and Part 11.3).

Part 0 — How to Use This Handbook

Data Analytics & Business Intelligence Department — Training, Engineering and Client Delivery Operating System

Document control	
Document ID	VEL-DA-001 (companion to VEL-DH-001, the company Delivery Handbook)
Version	1.0 — Module 1 (Foundation). Later modules are issued one at a time (see Part 13).
Owner	V. LOKESHWARAN, Founder
Audience	New employees with basic computer knowledge, plus the Head of Data & Analytics who trains them
Review cycle	Every quarter, and after every client project closes
Contact	velsoftwaresolutions.in@gmail.com

0.1 What this handbook is

This is the single operating manual for the Data Analytics & BI department. It does two jobs at once: it **trains** a beginner into a professional analytics engineer, and it defines the **corporate process** used to deliver analytics projects to clients, including international clients, from the first enquiry (0%) to a signed-off, maintained solution (100%).

Nothing here assumes you already know the subject. Every technical word is explained the first time it appears. Where a number depends on a vendor price or a law, this handbook tells you to **verify it from the official source on the day you quote** instead of printing a figure that will go out of date.

0.2 What is in Module 1, and what comes next

Delivered now (Module 1)	Delivered later, one module at a time
Department charter and services (Part 1) Organisation structure and roles (Part 2) Employee onboarding plan (Part 3) Training roadmap, Levels 1–7 (Part 4) **90-day training schedule (Part 5)** Technology stack and tool selection (Part 6) 17-stage project lifecycle summary (Part 7) Teaching format and repository standard (Part 8) Commercial model (Part 9) Training projects and learning sources (Parts 10–11) **First practical assignment A1 with real code (Part 12)** Module roadmap (Part 13), Annexes A–C	Module 2: Level 1 Data Fundamentals (full lesson) Module 3: Level 2 Excel Module 4: Level 3 SQL Module 5: Level 4 Python Module 6: Level 5 Business Intelligence Module 7: Level 6 Data Engineering Module 8: Level 7 Predictive Analytics Modules 9–12: Stages 01–17 in full depth, with templates Module 13: Internal portal Module 14: Sales kit (questionnaires, proposals, SOW)

Honest scope note

A 90-day plan can take a motivated beginner to **supervised junior-analyst level** across Levels 1–5, with a working introduction to Levels 6–7. It cannot produce an independent senior engineer; that takes roughly 6–12 months of real project work. This handbook plans for that reality instead of promising otherwise.

0.3 The twelve golden rules of the department

1. **Never assume the client's data is correct.** Audit it first (Stage 04). Every project starts with a data-quality assessment.
2. **Sell business outcomes, not charts.** A dashboard is a means; a trusted, verified management report is the product.

3. **One definition per KPI.** If two people calculate "revenue" differently, the dashboard is wrong even if the maths is right.
4. **Reconcile before you publish.** Important numbers must match a trusted source record, with evidence saved.
5. **Never commit client data, personal data, passwords or API keys to Git.** Not once, not temporarily.
6. **Predictions are estimates, not promises.** Always show a baseline, the error, and the limitations.
7. **Choose the simplest tool that works.** Excel before Power BI, a SQL view before a pipeline, a chart before a model.
8. **Write it down.** If it is not in a document or ticket, it did not happen.
9. **Do not move forward until the previous level is understood.** Gates are real; a failed gate means repeat, not skip.
10. **Small steps, saved often.** Commit whenever something works; keep every transformation reproducible.
11. **Ask early.** Being stuck for 30 minutes without asking is a mistake, not a virtue.
12. **Client data belongs to the client.** Treat it as if it were your own bank details; use the minimum needed, for the agreed purpose only.

0.4 How to use the handbook day to day

1. Find today's week in the 90-day schedule (Part 5). Read the week's objectives before touching a tool.
2. Follow the 15-step teaching format (Part 8) for each topic. Do the practical assignment, not just the reading.
3. Save every deliverable in the named folder. Update your daily log (template in Annex B).
4. At the end of each level, take the gate assessment. Pass mark and rules are in Part 4.
5. If anything in this book is wrong, unclear or out of date, tell the Head of Data & Analytics; they will fix the book. Improving it is part of everyone's job.

Part 1 — Department Charter and Commercial Services

1.1 Mission

To turn clients' scattered, unverified business data into **trusted decisions**: one verified reporting environment per client, delivered on time, documented, secure, and maintained on a monthly service.

1.2 The five service lines

Line	What the client receives	Typical outputs	Recurring revenue?
A. Data Analytics	Answers to business questions: where is revenue leaking, who are our best customers, why did margin fall?	Analysis report, KPI pack, customer segments, trend and root-cause findings	Sometimes (retainer)
B. Business Intelligence	Interactive dashboards and automated reports that leaders use every week	Power BI / Tableau / Looker Studio dashboards, scheduled reports, KPI monitor	Yes (maintenance)
C. Data Engineering	A reliable, clean, integrated data foundation the dashboards sit on	ETL/ELT pipelines, database or warehouse, data model, quality monitors	Yes (pipeline support)
D. Predictive Analytics	Forecasts and risk scores where history and volume justify them	Sales/demand/revenue/inventory forecast, churn model, anomaly alerts, maintenance prediction	Yes (model monitoring)
E. Managed Analytics	VEL runs the analytics function for the client month after month	Monthly reporting, dashboard upkeep, pipeline monitoring, KPI tracking, consulting	Yes (core recurring)

1.3 Selling outcomes, not dashboards

Do not sell	Sell instead
"A Power BI dashboard"	"A management reporting solution that consolidates sales, finance and operations data into one verified reporting environment, so the owner sees the same numbers as the accountant."
"A churn model"	"An early-warning list of customers likely to leave next quarter, so the account team can act, measured against what happens without the model."
"ETL pipelines"	"Reports that refresh themselves every morning, so nobody spends two days a month copying Excel sheets."

1.4 Target industries and typical first questions

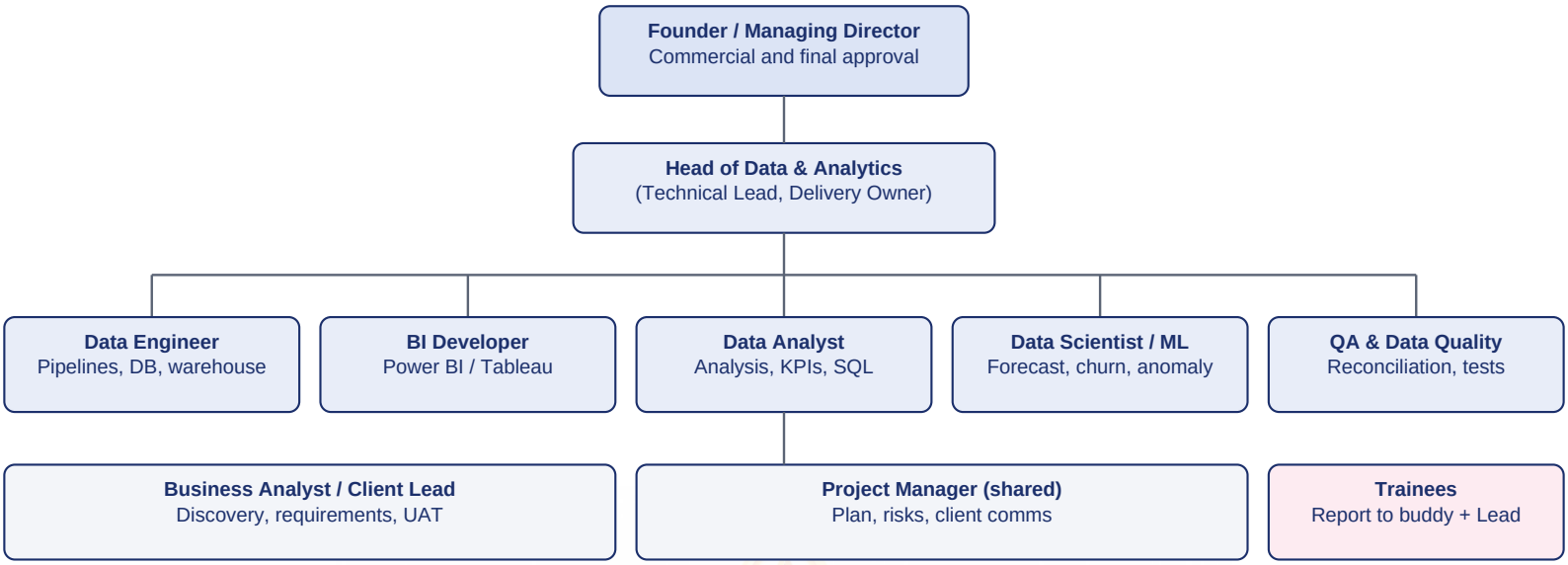
Industry	Typical systems	Typical first KPIs
Healthcare	HIS/EMR, billing, lab systems (**health data is highly sensitive**)	Bed occupancy, wait time, claim rejection rate, revenue per department
Manufacturing	ERP, MES, machine logs, Excel	OEE, scrap rate, on-time delivery, cost per unit, downtime
Retail	POS, inventory, loyalty, Excel	Sales per store, basket size, stock-outs, margin by category
E-commerce	Shopify/WooCommerce, ad platforms, payment gateways	Conversion rate, CAC, repeat-purchase rate, return rate, AOV
Logistics	TMS, GPS, warehouse system	On-time-in-full, cost per delivery, route utilisation, dwell time
Finance	Core banking/ledger, CRM, collections	Portfolio at risk, collection efficiency, cost-to-income, delinquency
SaaS	Product database, billing, CRM, support	MRR/ARR, churn, expansion revenue, activation, support load
Professional services	Time tracking, invoicing, CRM	Utilisation, realisation, project margin, pipeline, debtor days

Regulated industries

Healthcare and finance data carry legal duties (for example India's Digital Personal Data Protection Act 2023, and for overseas clients GDPR or HIPAA). Never accept a regulated-data project until the Founder and the client's data owner have agreed the legal basis, storage location and access rules in writing. When unsure, escalate; do not decide alone.

Part 2 — Organisation Structure

2.1 Department chart



A startup rarely has all of these as separate people. The chart shows the **roles**; Part 2.3 shows how to staff them in phases. One person may hold several roles in Phase 1, but the **duties must still be performed**, especially QA, which should never be marked by the person who built the work.

2.2 Roles in plain words

Role	What they really do	Key outputs	You ask them when
Head of Data & Analytics	Owns delivery quality, architecture decisions and training; reviews everything before the client sees it	Architecture, estimates, code review, gate sign-off	You are technically stuck or unsure about a design
Business Analyst / Client Lead	Talks to the client; turns business problems into written requirements and KPI definitions	Discovery report, PRD, KPI definitions, UAT plan	You do not understand what the client wants
Data Engineer	Gets data from sources, cleans it, loads it into a database or warehouse reliably	Pipelines, schemas, data model, monitoring	Data is missing, late, duplicated or slow
BI Developer	Builds dashboards and semantic models; writes DAX and measures	Dashboards, measures, RLS, refresh schedule	A number on a dashboard looks wrong
Data Analyst	Explores data, answers questions, writes SQL, finds insights	Analysis notes, KPI results, ad-hoc queries	You need an answer from the data
Data Scientist / ML Engineer	Builds and evaluates forecasts and classifiers; says when a model is **not** justified	Baseline, model, evaluation report, monitoring	Someone asks "can we predict X?"
QA & Data Quality Analyst	Reconciles dashboards to source, tests pipelines, checks access rules	Test evidence, reconciliation sheet, defect log	You want to know how to test something
Project Manager	Plan, risk, scope control, status, client communication	Plan, weekly status, change log, invoices trigger	Priorities, deadlines, scope questions
Trainee	Learns through the 90-day plan; works only on synthetic or authorised data until cleared	Daily log, assignments, gate results	—

2.3 Phased staffing for a startup

Phase	Headcount (indicative)	Who covers what	Trigger to move on
1. Foundation (months 0–6)	Founder + 1 Head/Lead + 2–3 trainees	Lead is analyst + engineer + BI developer; Founder does sales and project management; a **different trainee QA-checks** each deliverable	2 paid projects delivered and accepted
2. Delivery team (months 6–12)	+ 1 BI developer, + 1 data engineer, + part-time QA	Roles split; Lead reviews, does not build	Recurring revenue covers team salaries
3. Practice (year 2+)	+ Data scientist, + BA/Client lead, + support analyst	Managed analytics team runs monthly cycles	Defined by Founder with finance

2.4 Responsibility matrix for the standard delivery

R = does the work, A = accountable (one person only), C = consulted, I = informed.

Activity	Founder	Head	BA	Eng.	BI Dev	Analyst	QA	PM
Lead qualification	A	C	R	—	—	—	—	I
Discovery and requirements	I	A	R	C	C	C	I	C
Data audit	I	A	C	R	—	R	C	I
Architecture and estimate	A (price)	R	C	R	C	—	—	C
Pipelines and data model	—	A	C	R	C	C	C	I
Dashboards	—	A	C	C	R	C	C	I
Predictive model	—	A	C	C	—	C	C	I
QA and reconciliation	I	A	C	C	C	C	R	I
Deployment approval	A	R	I	R	R	—	C	I
Handover and acceptance	A	R	R	C	C	C	C	R

2.5 Career ladder

Level	Title	Proves by	Typical time
T	Trainee	Passes 90-day gates 1–6	0–3 months
J1	Junior Analyst / Junior BI Developer	Delivers a supervised module of a real project; passes QA with few defects	3–9 months
J2	Analyst / BI Developer / Data Engineer	Owns one dashboard or pipeline end to end; writes the data dictionary and test evidence	9–24 months
S	Senior (Architect, Lead)	Designs solutions, estimates cost, leads client discovery, reviews others	24+ months

Part 3 — Employee Onboarding Plan

3.1 Principles

- Every new employee gets a **buddy** (a colleague who answers questions) and a **mentor** (the Head of Data & Analytics).
- The employee works **only on synthetic or explicitly authorised data** until the security briefing is signed.
- Equipment and accounts are ready **before day 1**. A new joiner should never wait on IT.
- Success is measured by gates passed, not hours sat.

3.2 Before day 1 (Head of Data & Analytics and Founder)

#	Action	Owner	Done
1	Employment agreement, NDA and data-protection undertaking signed	Founder	[ ]
2	Laptop prepared: Windows 10/11 or Linux, 16 GB RAM recommended (8 GB minimum), 256 GB free storage, disk encryption on	Head	[ ]
3	Company email account created; multi-factor authentication (MFA) enforced	Founder	[ ]
4	GitHub account invited to the company organisation (private repos only)	Head	[ ]
5	Buddy assigned; first-week calendar sent	Head	[ ]
6	Training repository created from the template (Part 8) with synthetic data only	Head	[ ]
7	Password manager seat assigned (shared secrets live only there)	Founder	[ ]

3.3 Day 1 — Welcome and safety

Time	Activity
09:30–10:30	Welcome by the Founder: company mission, services, the 12 golden rules (Part 0)
10:30–11:30	Tour of this handbook and the 90-day plan; meet buddy and mentor
11:30–13:00	**Information-security briefing**: MFA, password manager, no client data on personal devices or personal cloud, no secrets in Git, how to report an incident. Employee signs the acknowledgement.
14:00–17:00	Workstation setup using the checklist in 3.5; verify every tool with the listed command
17:00–17:30	First daily log written (Annex B) and reviewed by buddy

3.4 Week 1 outline (detail in Part 5)

Day	Focus	Output
1	Orientation, security briefing, workstation	Verified setup log
2	Level 1: what data is; databases vs spreadsheets; rows, columns, data types	Notes + glossary (20 terms)
3	Level 1: keys, relationships, data quality dimensions	Hand-drawn table diagram of a shop database
4	Level 1: data lifecycle, KPIs, governance. Start Assignment A1 (Part 12)	A1 files generated
5	Finish A1; Level 1 gate quiz; Friday review with mentor	A1 audit report + quiz result

3.5 Workstation setup checklist

Install in this order. After each tool, run the **verify command** and paste the output into your setup log. Download tools only from the official sites listed; never from third-party "download" pages.

Tool	Official source	Verify command / action	Needed from
Git	git-scm.com	`git --version`	Week 1
VS Code	code.visualstudio.com	Open VS Code; install the Python and SQLTools extensions	Week 1
Python 3.11 or newer	python.org/downloads	`python --version` (on Linux/macOS: `python3 --version`)	Week 1
Microsoft Excel (licensed) or Google Sheets	company licence / sheets.google.com	Open a workbook; check that Power Query (Data tab, Get Data) exists	Week 2
PostgreSQL 16 or newer + pgAdmin	postgresql.org/download	`psql --version`; connect locally	Week 4
Power BI Desktop (Windows only)	Microsoft Store or Microsoft download page	Open and sign in with the work account	Week 8
Draw.io (diagrams.net)	diagrams.net	Create and save a test diagram as .drawio	Week 1

> **Power BI on Mac or Linux**
> Power BI Desktop runs on Windows only. A trainee on another system uses a company Windows virtual machine or loaner laptop for Weeks 8–9. Plan this in advance; check Microsoft Learn for current options.

Python project environment (do this once per project)

```
# Windows (PowerShell)
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install pandas numpy matplotlib openpyxl requests

# Linux / macOS
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install pandas numpy matplotlib openpyxl requests

# Record exactly what you installed, so the next person can repeat it
pip freeze > requirements.txt
```

Why a virtual environment? It keeps each project's libraries separate, so upgrading one project never breaks another. **Common error:** *"python is not recognised"* means Python was installed without "Add python.exe to PATH"; re-run the installer and tick that box.

3.6 First-month checkpoints

When	Checkpoint	Pass condition
End of week 1	Orientation review	Security acknowledgement signed; setup verified; A1 submitted
End of week 3	Gate 1 (Levels 1–2)	Written 70%+ and Excel practical rated 3 of 5 or better on every rubric line
Day 30	Mentor 1-to-1	Daily logs complete; buddy feedback; plan adjustments agreed in writing
Day 90	Probation review	Gates 1–6 passed; Projects 1–6 reviewed; recommendation to Founder

Part 4 — Training Roadmap: Levels 1 to 7

Seven levels, taught strictly in order. Each level ends with a **gate**: a written test and a practical task marked against a rubric. Do not start the next level until the gate is passed.



Level	Topics (all taught from zero)	Hours	Proof of learning
1 Data Fundamentals	What is data; structured, semi-structured, unstructured; databases vs spreadsheets; tables, rows, columns; data types; primary and foreign keys; data quality; data lifecycle; business KPIs; data governance	25	Assignment A1 (data audit) + quiz
2 Excel	Entry and formatting; sorting, filtering; IF, SUMIF, COUNTIF; XLOOKUP; PivotTables; charts; data validation; Power Query; dashboard basics	45	Project 1: Excel business dashboard
3 SQL	Database basics; SELECT, INSERT, UPDATE, DELETE; WHERE, ORDER BY; GROUP BY, HAVING; JOINs; subqueries; CTEs; window functions; views; indexes; optimisation; database security	55	Project 2: SQL sales analysis (30 queries)
4 Python	Fundamentals; variables, lists, dicts; functions; file handling; pandas; NumPy; Matplotlib; cleaning; transformation; API extraction; error handling; automated reporting	60	Project 3: cleaning project with tests
5 Business Intelligence	Power BI Desktop; Power Query; data modelling; relationships; DAX; measures vs calculated columns; dashboard design; filters; row-level security; scheduled refresh; Power BI Service; Tableau fundamentals and alternatives	60	Project 4: executive dashboard
6 Data Engineering	ETL vs ELT; pipelines; API integration; PostgreSQL; warehouses; star and snowflake schemas; fact and dimension tables; incremental loading; validation; scheduling; failure handling	55	Projects 5 and 6: API-to-DB pipeline; automated monthly report
7 Predictive Analytics	Statistics; probability; correlation; regression; classification; time-series; clustering; feature engineering; evaluation; overfitting; data leakage; monitoring; **when NOT to predict**	50	Project 7: forecast vs baseline; then Project 8

Hours are guided learning plus practice. Add review and assessment time. Total about 350 hours, or about 9 weeks of full-time study; the rest of the 13 weeks is projects, gates, revision and client-process training.

4.1 Gate rules

- **Written test:** pass mark 70%. Open-book for syntax, closed-book for concepts.
- **Practical task:** rubric of 5 lines scored 0–5. Every line must score 3 or more and the average must be 3.5 or more.
- **Fail:** mentor writes a gap list; trainee repeats only the weak topics for up to one week; one retake. A second failure triggers a mentor and Founder review of the plan.
- Assessors mark the **evidence** (files, queries, screenshots) not the explanation given in person.

4.2 When predictive analytics is appropriate, and when it is not

Predictive work is the most over-sold service in analytics. Use this decision table before proposing it:

Question	If yes	If no
Is there a clear decision that a prediction will change? (e.g. order more stock, call a customer)	Continue	Do a dashboard or analysis instead
Is there enough clean history? (guide: at least 2 full seasonal cycles for forecasting; hundreds of examples of the rare class for classification)	Continue	Collect data first; offer descriptive analytics now
Can a simple baseline (last year's value, moving average, average rate) be beaten by a clear margin?	Model is justified	Use the baseline and say so honestly
Will the pattern stay stable? (not disrupted by price changes, new products, one-off events)	Continue	Warn the client; add monitoring and shorter horizons
Can the client act on it and measure the result?	Deploy after approval	Do not deploy; document findings only

Never promise outcomes

Predictions are probabilistic estimates with error. Every deliverable must show the baseline, the error measure on unseen data, the assumptions, and the sentence: "Forecasts are estimates and not guaranteed outcomes."

Part 5 — The 90-Day Training Schedule

Rhythm: 13 weeks (65 working days). Each day: **09:30–10:00** stand-up and plan; **10:00–13:00** learn (read, watch, type along); **14:00–16:30** practice and assignment; **16:30–17:30** review with buddy, daily log, commit work to Git. **Friday afternoon:** mentor review and next-week preview.

Wk	Level and focus	Key activities	Deliverable	Gate
1	Orientation + **L1 Data Fundamentals**	Security briefing; workstation; data concepts; keys; data quality; lifecycle; KPIs; governance	**Assignment A1** audit report; setup log	Quiz
2	**L2 Excel I**	Entry, formats, sorting, filtering; IF, SUMIF, COUNTIF; XLOOKUP; data validation	Cleaned sales workbook	—
3	**L2 Excel II**	PivotTables; charts; Power Query; dashboard layout and slicers	**Project 1** Excel dashboard	**Gate 1** (L1–2)
4	**L3 SQL I**	Install PostgreSQL; create tables; SELECT/WHERE/ORDER BY; INSERT/UPDATE/DELETE; GROUP BY/HAVING; JOINs	15 queries + schema script	—
5	**L3 SQL II**	Subqueries; CTEs; window functions; views; indexes; EXPLAIN; roles and permissions	**Project 2** SQL sales analysis (30 queries)	**Gate 2** (L3)
6	**L4 Python I**	Syntax; lists/dicts; functions; files; errors; VS Code; Git basics	20 practice scripts in repo	—
7	**L4 Python II**	pandas; NumPy; cleaning; merging; Matplotlib; REST API with requests; reports	**Project 3** data-cleaning pipeline + unit tests	**Gate 3** (L4)
8	**L5 BI I**	Power BI Desktop; Power Query; star-schema model; relationships; first report	Model + 3-page report	—
9	**L5 BI II**	DAX measures; time intelligence; design and accessibility; RLS; Service; scheduled refresh; Tableau tour	**Project 4** executive dashboard (published to test workspace)	**Gate 4** (L5)
10	**L6 Data Eng I**	ETL vs ELT; warehouse design; fact and dimension tables; PostgreSQL loading; validation rules	**Project 5** API-to-database pipeline	—
11	**L6 Data Eng II**	Incremental loads; scheduling; retries; logging; alerts; failure drills	**Project 6** automated monthly report	**Gate 5** (L6)
12	**L7 Predictive**	Statistics; baselines; regression; time-series split; evaluation; leakage; limitations	Project 7 started: forecast vs baseline	—
13	Client process + review	Stages 01–17 walkthrough; mock discovery call; SOW reading; mock handover; portfolio review	Project 7 report; Project 8 plan approved	**Gate 6** panel + probation review

> **Days 91–150 (Phase 2)**
> Project 7 (forecasting) is finished and Project 8, the full simulated client engagement, is run across Stages 01–17 under supervision. A trainee is not scheduled onto a paid client project before Project 8 is accepted, except as a shadow.

5.1 Daily habits that make the plan work

- **Daily log** (template in Annex B): what I did, what I learned, what blocked me, what is next. Five lines, no more.
- **Commit every day**: at least one commit with a clear message. Message format: `L3.05: add window-function queries` (level.task: message).

- **Explain it aloud**: end each topic by explaining it to your buddy in two minutes. If you cannot, you do not yet understand it.
- **Break it on purpose**: after something works, change an input and see how it fails. This is how errors become familiar, not frightening.

5.2 Weekly review (Friday, 45 minutes, mentor)

1. Show the week's deliverable running, not slides.
2. Mentor asks three "why" questions about the work; answers are noted.
3. Defects found are logged with owner and fix date.
4. Plan for next week confirmed; any slippage handled by cutting scope, not by skipping tests.

Part 6 — Technology Stack and Selection

6.1 The stack VEL teaches

Category	Tools	Used for
Spreadsheet	Microsoft Excel, Google Sheets	Small clients, prototypes, requirement workshops, Level 2
BI	Power BI, Tableau, Looker Studio	Dashboards and reports; Power BI is the primary tool
Programming	Python	Cleaning, API extraction, automation, modelling
Analysis	pandas, NumPy	Tabular data processing, numerical work
Visualisation	Matplotlib, Plotly	Static charts for reports, interactive charts
Database	PostgreSQL, MySQL	Operational stores, staging and small warehouses
Warehouse	BigQuery, Snowflake, Redshift	Larger volume and many users; usage-based cost
ETL / transformation	dbt, Airflow, Power Query	SQL transformations; scheduled pipelines; light self-service shaping
APIs	REST APIs	Pulling data from SaaS, ERP and CRM systems
Version control	Git, GitHub	History, review, collaboration (private repositories)
Development	VS Code	Editor for Python, SQL, Markdown
Cloud	AWS, Azure, GCP	Hosting databases, storage, schedulers
Machine learning	scikit-learn	Classical models: regression, classification, clustering
Documentation	Markdown, Draw.io	Data dictionary, architecture and lineage diagrams

6.2 How to choose tools for a client

Choose by evidence, in this order, and write the reasons into the architecture document (Stage 05):

Factor	Question to ask the client	Effect on choice
Existing ecosystem	Do they already use Microsoft 365, Google Workspace, or neither?	Microsoft 365 users: Power BI is natural. Google-centric: Looker Studio is strong and cheap.
Budget	What can they spend per month on licences and cloud?	Low budget: Excel + Looker Studio + PostgreSQL. Verify licence prices on the day you quote.
Data volume	Rows per table now and in 2 years? Refresh frequency?	Under a few million rows: PostgreSQL is enough. Tens of millions and growing: consider a cloud warehouse.
Users	How many viewers and creators? Mobile? External sharing?	Per-user licences scale with viewers; check Power BI licensing rules for sharing and capacity.
Licensing	Is any tool restricted by their policy or by data-residency law?	Record licence terms; open-source preferred where equal.
Maintenance	Who looks after it after handover? Skills they have?	Prefer fewer moving parts; a simple stack the client can own beats a clever one only VEL can run.
Security	Sensitive data? Required hosting region? Audit needs?	May force on-premises or a specific cloud region, and row-level security.

Verify before quoting

Vendor prices, licence tiers and free-tier limits change often. Before any proposal, open the vendor's official pricing page, record the date, URL and the plan name in the cost sheet, and attach a screenshot. Never copy a price from memory, a blog or this handbook.

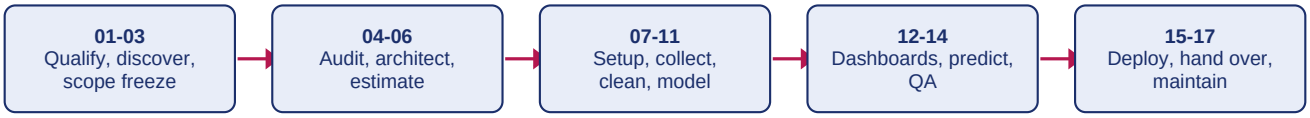
6.3 Reference architecture (default)



Each arrow is a logged, repeatable step. **Raw data is never edited**; fixes are made in the clean layer so you can always trace a dashboard number back to the original record (*data lineage*).

Part 7 — The 17-Stage Client Project Lifecycle

Every client project passes through these stages in order. Each stage ends with a named output and a sign-off. Full templates and step-by-step lessons are released in Modules 9–12; this is the map.



Stage	Name	Main output	Typical owner	Sign-off
01	Lead qualification	Lead qualification report (industry, problem, data, users, budget, decision-maker)	BA / Founder	Founder
02	Client discovery	Discovery report: sources, ERP/CRM, processes, KPIs, data owners, quality and security issues	BA	Head
03	Requirements and scope freeze	Approved PRD and Statement of Work with deliverables, exclusions, timeline, acceptance criteria	BA + PM	Client + Founder
04	Data audit	Data quality assessment (completeness, duplicates, formats, outliers, definitions, freshness, access)	Analyst / Eng.	Head
05	Analytics architecture	Architecture and data-flow diagrams; tool selection rationale	Head	Founder
06	Cost estimation	One-time and recurring cost model with sourced prices and assumptions	Head + PM	Founder
07	Development environment	Configured tools, cloud workspace, access	Eng.	Head
08	Repository structure	Standard repo (Part 8), retention and access rules	Eng.	Head
09	Collection and integration	Working connectors with authentication, pagination, retries, rate-limit handling, logging	Eng.	Head
10	Cleaning and transformation	Cleaned data with traceability to originals; rule log	Eng. / Analyst	QA
11	Data modelling	Fact and dimension tables, measures, KPI definitions, dictionary; client validates calculations	Eng. / BI	Client
12	Dashboard development	Dashboards with filters, drill-down, comparisons, export, user-specific access	BI Dev	QA + Client
13	Predictive analytics (if justified)	Objective, baseline, evaluation, limitations; deploy only after approval	Data Sci.	Head + Client
14	QA and validation	Test evidence; reconciliation to trusted source records	QA	Head
15	Deployment	Production database, pipelines, published dashboards, refresh, monitoring, backup	Eng. + BI	Client + Founder
16	Demonstration and handover	User guide, refresh and access guides, training, final acceptance record	PM + BA	Client
17	Ongoing maintenance	Monthly checks, updates, monitoring, reporting, support	Managed team	Monthly report

Hard stops

No Stage 05 before Stage 04 is complete. No Stage 15 without written production approval. No client dataset leaves the approved environment at any stage. No dashboard is published until key metrics are reconciled (Stage 14).

Part 8 — Teaching Format and Repository Standard

8.1 The 15-step format used for every topic

Every lesson in every module is written in this order, so a beginner always knows where they are. A topic is **complete** only when steps 10, 14 and 15 are evidenced.

#	Step	What it contains
1	Definition	One plain sentence; no unexplained jargon
2	Business purpose	Why a client would pay for this
3	Real-world example	A small business story using realistic (synthetic) numbers
4	Required software	Exact tools and versions
5	Installation	Official download source and commands
6	Configuration	Settings, folders, environments
7	Step-by-step implementation	Numbered actions a beginner can follow
8	Code and commands	Copy-ready, commented, tested
9	Expected output	What success looks like, pasted from a real run
10	Verification	A check that proves it worked (count, total, test)
11	Common errors	Message, cause, fix
12	Security considerations	Data handling, credentials, permissions
13	Documentation	What to write in README / data dictionary
14	Practical assignment	A task with a clear deliverable
15	Assessment	Questions and rubric; pass condition

8.2 Standard project repository

```
data-analytics-project/
  README.md  requirements.txt  .gitignore
  docs/        project-overview, requirements, data-dictionary, kpi-definitions,
               architecture, data-lineage, user-guide  (all .md)
  data/        sample/  raw/  processed/  reference/
  src/         extraction/ cleaning/ transformation/ analytics/ reporting/
  sql/         schemas/ queries/ views/ procedures/
  pipelines/   ingestion/ transformation/ orchestration/
  dashboards/  powerbi/ tableau/
  models/      training/ evaluation/ inference/
  tests/       data_quality/ unit/ integration/
  deployment/
```

Adapt the structure to the project (a small Excel-only job does not need `models/`), but keep the names so any VEL engineer can find things. Document every change from the standard in `docs/project-overview.md`.

8.3 Data handling rules (mandatory)

Rule	How it is enforced
Client datasets, personal data, credentials and confidential exports **never** go into a public repository, and never into Git history at all	`.gitignore` excludes `data/raw/`, `data/processed/`, `*.pbix` with embedded data, `.env`; run the secret scan before every push
Secrets live in a password manager or environment variables	Code reads `os.environ["DB_PASSWORD"]`; the repo holds only `.env.example` with fake values
Only the minimum personal data is collected	Drop or hash columns such as phone, email, national ID unless the PRD requires them
Retention	Agree per project in the SOW (default: raw client extracts deleted or archived 90 days after final acceptance unless the contract says otherwise)
Access	Least privilege: read-only database users for reporting; named accounts; MFA; access reviewed monthly under managed service
Training uses synthetic data	Real client data is allowed in training only after written client consent and Head approval

```
# .gitignore (minimum)
.venv/
__pycache__/
.env
data/raw/
data/processed/
*.csv.gz
*.xlsx#
~$*.xlsx
```

Part 9 — Commercial Model and Pricing

9.1 The formulas

Total Project Cost = Labour + Software licences + Infrastructure + Data engineering + Dashboard development + QA + Project management + Other direct costs. (Labour is the hours of each role at that role's fully loaded cost rate; listing engineering, dashboard, QA and PM separately keeps the estimate auditable.)

Selling price = (Delivery cost + Risk contingency + Overhead) / (1 – Target profit margin).

Gross margin = (Price – Direct delivery cost) / Price. **Break-even** (monthly) = Fixed monthly overhead / Contribution per recurring contract, where contribution = monthly fee – monthly cost to serve.

9.2 Worked example (ILLUSTRATIVE numbers, not VEL rates)

Small retail client, one executive dashboard, three data sources, one nightly pipeline. Rates and percentages below are **assumptions** chosen to show the arithmetic; replace them with VEL's approved rate card and verified vendor prices.

Line	Hours	Rate (INR/h, assumed)	Cost (INR)
Discovery and requirements	24	800	19,200
Data audit	24	800	19,200
Data engineering	80	800	64,000
Dashboard development	60	800	48,000
QA and reconciliation	30	800	24,000
Project management	24	800	19,200
Labour subtotal	242		**1,93,600**
Dev/test cloud and tools (VEL cost)			12,000
Other direct costs			5,000
Direct delivery cost			**2,10,600**
Risk contingency 15%			31,590
Overhead 20% of (cost + contingency)			48,438
Full cost			**2,90,628**
Price at 30% target margin: 2,90,628 / 0.70			4,15,183 (quote INR 4,15,000)

Check: gross margin = (4,15,000 – 2,10,600) / 4,15,000 = **49.3%**. If the contingency is fully used, profit is 4,15,000 – 2,90,628 = INR 1,24,372, a net margin of **30.0%**.

Recurring example	Value
Monthly maintenance fee (assumed)	INR 25,000
Monthly cost to serve (monitoring, checks, support hours; assumed)	INR 9,000
Contribution per contract	INR 16,000
Fixed monthly overhead of the department (assumed)	INR 1,50,000
Break-even contracts = 1,50,000 / 16,000	9.4, so 10 contracts

9.3 Keep two things separate on every quote

VEL service fees (VEL invoices)	Third-party subscriptions (client pays vendor, or VEL passes through at cost with written approval)
Discovery, build, QA, training, managed service, support	Power BI / Tableau licences, cloud database and storage, warehouse compute, API plans, backup storage
Fixed price per SOW or monthly retainer	Usage-based: can rise with data volume; state the assumption and a review date in the SOW

Also compute and record for each project: **cost per dashboard** (dashboard development + its share of QA and PM / number of dashboards), **cost per pipeline**, **cost per customer** (total delivery and support cost / active clients), **recurring revenue** (sum of monthly fees) and **licence and cloud expenses** as separate lines.

Pricing sources

Cloud and BI prices must come from the vendors' official pricing pages (Part 11.3), captured with date, region, plan and currency. Record exchange rates and tax treatment (for example GST for Indian clients; export-of-services rules for overseas clients — confirm with the company's accountant).

Part 10 — Mandatory Training Projects

All projects use **synthetic or explicitly authorised data**. Each has a repository, a README, a data dictionary and a reviewed deliverable. Acceptance is by the Head of Data & Analytics against the rubric.

#	Project	Skills proved	Acceptance criteria	Week
1	Excel business dashboard	Formulas, XLOOKUP, PivotTables, charts, validation, Power Query	KPIs reconcile to raw totals; refreshable; documented; no hard-coded numbers in charts	3
2	SQL sales analysis	Joins, aggregation, CTEs, windows, views	30 queries, each with a business question and a verified result; schema script re-runs cleanly	5
3	Python data cleaning	pandas, validation, logging, tests	Rule log; before/after counts; 10+ unit tests pass; raw file untouched	7
4	Power BI executive dashboard	Model, DAX, design, RLS, refresh	Star schema; 10 measures documented; RLS tested with two roles; accessibility check	9
5	API-to-database pipeline	Auth, pagination, retries, loading	Idempotent re-run; failure handled and logged; rate limits respected	10
6	Automated monthly reporting	Scheduling, templating, delivery	Report generated with no manual step; reconciliation row included; failure alert works	11
7	Predictive sales forecast	Time-series split, baseline, evaluation	Beats baseline on held-out data or honestly says it does not; limitations page; no leakage	12–13 +
8	Full simulated client project	Stages 01–17	All stage outputs produced; mock client acceptance record signed	Days 91–150

Part 11 — Official Learning Sources

Use **current official documentation** first. Third-party tutorials may supplement but never replace it. URLs below were supplied by the company brief or are the vendors' main documentation sites; if a link has moved, search from the vendor's home page and tell the Head so this list is corrected.

11.1 Core learning sources

Topic	Source	URL
Microsoft training (all products)	Microsoft Learn	https://learn.microsoft.com/
Power BI	Microsoft Learn: Power BI	https://learn.microsoft.com/power-bi/
Tableau	Tableau Learn	https://www.tableau.com/learn
Python	Python documentation	https://docs.python.org/3/
pandas	pandas documentation	https://pandas.pydata.org/docs/
PostgreSQL	PostgreSQL documentation	https://www.postgresql.org/docs/
Google Cloud	Google Cloud documentation	https://cloud.google.com/docs
AWS	AWS documentation	https://docs.aws.amazon.com/
dbt	dbt documentation	https://docs.getdbt.com/
scikit-learn	scikit-learn user guide	https://scikit-learn.org/stable/

11.2 Supporting official sources by level

Level	Also use	URL
1 Data fundamentals	Microsoft Learn data-fundamentals paths; PostgreSQL tutorial	https://learn.microsoft.com/ ; https://www.postgresql.org/docs/current/tutorial.html
2 Excel	Microsoft Support: Excel help and training	https://support.microsoft.com/excel
3 SQL	PostgreSQL SQL language reference	https://www.postgresql.org/docs/current/sql.html
4 Python	NumPy, Matplotlib, Plotly, Requests	https://numpy.org/doc/ ; https://matplotlib.org/stable/ ; https://plotly.com/python/ ; https://requests.readthedocs.io/
5 BI	DAX reference; Looker Studio help	https://learn.microsoft.com/dax/ ; https://cloud.google.com/looker-studio
6 Data engineering	dbt; Apache Airflow; cloud warehouse docs	https://docs.getdbt.com/ ; https://airflow.apache.org/docs/ ; https://cloud.google.com/bigquery/docs ; https://docs.snowflake.com/ ; https://docs.aws.amazon.com/redshift/
7 Predictive	scikit-learn model evaluation and cross-validation	https://scikit-learn.org/stable/model_selection.html
Tools	Git, VS Code, GitHub	https://git-scm.com/doc ; https://code.visualstudio.com/docs ; https://docs.github.com/
Azure	Azure documentation	https://learn.microsoft.com/azure/

11.3 Official pricing and licensing pages (verify on the day you quote)

Vendor	Pricing / licensing page
Power BI	https://powerbi.microsoft.com/pricing/
Tableau	https://www.tableau.com/pricing
AWS	https://aws.amazon.com/pricing/
Microsoft Azure	https://azure.microsoft.com/pricing/
Google Cloud	https://cloud.google.com/pricing
Snowflake	https://www.snowflake.com/pricing/
Python, pandas, PostgreSQL, scikit-learn, dbt Core	Open-source licences; read each project's LICENSE file and record it in the project's third-party register

Source discipline
For every cost, rule or legal statement in a proposal, the cost sheet or document must record: source URL, date accessed, plan or version, and who verified it. A number without a source is an assumption and must be labelled as one.

Part 12 — First Practical Assignment (A1): Audit a Sales Dataset

Level 1 · Week 1 · Estimated time 6–8 hours · Data: synthetic (generated by the script below)

This assignment follows the 15-step teaching format (Part 8.1). It is deliberately small: the aim is to learn to **distrust data politely** and to prove, with evidence, what is wrong with it. This is Stage 04 in miniature.

1. Definition

A **data audit** is a systematic, read-only inspection of a dataset to find missing values, duplicates, wrong formats, impossible values and inconsistent definitions, before anyone builds a report on it.

2. Business purpose

A dashboard built on unaudited data shows confident wrong numbers. The audit is the cheapest place to find errors and the first thing a client sees that proves VEL is careful.

3. Real-world example

"Sunrise Retail" (fictional) exports 505 order lines from its billing software. The owner says January–June sales were "about right". Before building anything we check: are there duplicate orders (double counting revenue)? Negative quantities? Dates in two formats (so some months sort wrongly)? Regions typed in different cases (so a pivot shows "north" and "North" as two regions)? Each defect changes the totals.

4–6. Software, installation and configuration

- Python 3.11+, pandas, VS Code, Git (Part 3.5). Excel or Google Sheets for the manual check.
- Create the folder `a1-sales-audit/`, create and activate a virtual environment, run `pip install pandas`, then save the two scripts below into the folder.
- Use `git init` and add a `.gitignore` (Part 8.3). Even though this data is fake, practise the habit.

7. Step-by-step implementation

1. Run `generate_sales_data.py`. It writes `data/sales_raw.csv` (505 rows). The fixed seed 2505 means everybody gets the same file.
2. Open the CSV in Excel. Without code, write down by eye 3 problems you can spot. (Most are hidden; this shows why code is needed.)
3. Read `audit_data.py` line by line. For each of the six checks write one sentence: what does it test and why does it matter?
4. Run the audit script. Compare its numbers with your Excel observations.
5. Reproduce two of the checks in Excel: remove duplicates (note the count), and add a helper column `=F2*G2-H2` to find revenue mismatches.
6. Write the audit report (template in Annex A): findings table, business impact of each, recommended fix, and the question you would ask the data owner.
7. Commit with the message `L1.01: audit sales_raw.csv and report findings`.

8. Code

generate_sales_data.py (creates synthetic data with deliberate defects)

```
"""Generate the synthetic 'Sunrise Retail' sales dataset for Assignment A1.

All data is fake. A fixed seed makes the output identical on every machine.
Deliberate defects are injected so the trainee can practise a data audit.
"""
import csv
import random
from datetime import date, timedelta
from pathlib import Path

SEED = 2505
N_ROWS = 500
OUT = Path(__file__).parent / "data" / "sales_raw.csv"

REGIONS = ["North", "South", "East", "West"]
PRODUCTS = {"P01": ("Notebook", 120.0), "P02": ("Pen Pack", 60.0),
            "P03": ("Backpack", 850.0), "P04": ("Desk Lamp", 640.0),
            "P05": ("Water Bottle", 310.0)}


def build_rows(rng: random.Random) -> list[dict]:
    start = date(2025, 1, 1)
    rows = []
    for i in range(1, N_ROWS + 1):
        pid = rng.choice(list(PRODUCTS))
        qty = rng.randint(1, 10)
        price = PRODUCTS[pid][1]
        rows.append({
            "order_id": f"ORD{i:04d}",
            "order_date": (start + timedelta(days=rng.randint(0, 179))).isoformat(),
            "customer_id": f"C{rng.randint(1, 80):03d}",
            "region": rng.choice(REGIONS),
            "product_id": pid,
            "quantity": qty,
            "unit_price": price,
            "revenue": round(qty * price, 2),
        })
    return rows


def inject_defects(rows: list[dict], rng: random.Random) -> None:
    """Break the data on purpose (about 4% of rows)."""
    for r in rng.sample(rows, 10):
        r["region"] = ""                       # missing value
    for r in rng.sample(rows, 8):
        r["region"] = r["region"].lower() or "north"  # inconsistent case
```

```
    for r in rng.sample(rows, 5):
        r["quantity"] = -abs(int(r["quantity"]))      # impossible value
    for r in rng.sample(rows, 6):
        d = date.fromisoformat(r["order_date"])
        r["order_date"] = d.strftime("%d/%m/%Y")      # wrong date format
    for r in rng.sample(rows, 4):
        r["revenue"] = round(float(r["revenue"]) * 1.5, 2)  # wrong calculation
    rows.extend(dict(r) for r in rng.sample(rows, 5))        # duplicate rows


def main() -> None:
    rng = random.Random(SEED)
    rows = build_rows(rng)
    inject_defects(rows, rng)
    OUT.parent.mkdir(parents=True, exist_ok=True)
    with OUT.open("w", newline="", encoding="utf-8") as fh:
        w = csv.DictWriter(fh, fieldnames=list(rows[0]))
        w.writeheader()
        w.writerows(rows)
    print(f"Wrote {len(rows)} rows to {OUT}")


if __name__ == "__main__":
    main()
```

audit_data.py (read-only audit; never changes the source file)

```
"""Data-quality audit for sales_raw.csv (Stage 04 practice).

Usage:  python audit_data.py [path/to/file.csv]
Read-only: never edits the source file.
"""
import sys
from pathlib import Path

import pandas as pd

path = Path(sys.argv[1]) if len(sys.argv) > 1 else Path(__file__).parent / "data" / "sales_raw.csv"
df = pd.read_csv(path, dtype=str, keep_default_na=False)   # read everything as text first

print(f"File: {path.name}")
print(f"Rows: {len(df)}   Columns: {len(df.columns)}")

blank = {c: int((df[c].str.strip() == "").sum()) for c in df.columns}
print("\n1. Missing values (blank cells):")
for c, n in blank.items():
    if n:
        print(f"   {c}: {n}")

print(f"\n2. Fully duplicated rows: {int(df.duplicated().sum())}")
print(f"   Duplicate order_id values: {int(df['order_id'].duplicated().sum())}")

dates = pd.to_datetime(df["order_date"], format="%Y-%m-%d", errors="coerce")
print(f"\n3. order_date not in YYYY-MM-DD format: {int(dates.isna().sum())}")

qty = pd.to_numeric(df["quantity"], errors="coerce")
print(f"\n4. quantity <= 0: {int((qty <= 0).sum())}")

reg = df.loc[df["region"].str.strip() != "", "region"]
bad_case = int((reg != reg.str.title()).sum())
print(f"\n5. region values with inconsistent case: {bad_case}")

calc = (qty * pd.to_numeric(df["unit_price"], errors="coerce")).round(2)
mismatch = int((calc != pd.to_numeric(df["revenue"], errors="coerce")).sum())
print(f"\n6. revenue != quantity x unit_price: {mismatch}")

print("\nVerdict: DO NOT use for reporting until defects are resolved with the data owner.")
```

9. Expected output

Run on the file above, the real output of the audit script is:

```
File: sales_raw.csv
Rows: 505   Columns: 8

1. Missing values (blank cells):
   region: 10

2. Fully duplicated rows: 5
   Duplicate order_id values: 5

3. order_date not in YYYY-MM-DD format: 6

4. quantity <= 0: 5

5. region values with inconsistent case: 8

6. revenue != quantity x unit_price: 9

Verdict: DO NOT use for reporting until defects are resolved with the data owner.
```

The first rows of `sales_raw.csv` start with: `ORD0001, 2025-06-05, C066, North, P03, 8, 850.0, 6800.0`.

10. Verification

Check	How to prove it	Expected
Row count	`len(df)` or Excel status bar	505 data rows (500 orders + 5 duplicates)
Duplicates	Excel: Data > Remove Duplicates (all columns)	5 rows removed
Missing region	Excel filter on blanks	10 rows
Revenue mismatch	Helper column `=F2*G2-H2`	9 non-zero rows
Why 9, not the 4 injected?	Inspect the mismatching rows	4 rows had revenue inflated by 1.5x, and the 5 rows given a negative quantity no longer equal quantity x price, so both defect groups show up

11. Common errors

Message or symptom	Cause	Fix
`ModuleNotFoundError: No module named 'pandas'`	Virtual environment not active or pandas not installed	Activate `.venv`, run `pip install pandas`
`FileNotFoundError ... sales_raw.csv`	Generator not run, or run from another folder	Run the generator first; the scripts use paths relative to their own folder
Excel shows dates as `05/06/2025` for some rows and `2025-06-05` for others	Genuine data defect (and Excel locale guessing)	Report it; never fix by hand in the raw file
Counts differ from this book	A different seed or edited file	Re-generate with seed 2505; do not edit the CSV

12. Security considerations

- Treat the file as if it were real client data: do not email it or upload it to online tools.
- The audit is **read-only**. Never overwrite the raw file; later cleaning writes a new file in `data/processed/`.
- Real projects: a data-sharing agreement must exist before any file is received (Stage 02).

13. Documentation

Create `README.md` (what the project is, how to run both scripts) and `docs/data-dictionary.md` with one row per column: name, meaning, type, allowed values, and known issues. For example: *region — text — North/South/East/West — 10 blanks, 8 wrong-case.*

14. Practical assignment (deliverables)

1. `audit_report.md` or PDF (Annex A template), maximum 2 pages.
2. Both scripts, plus README and data dictionary, committed to Git.
3. **Extension (required):** add two new checks to `audit_data.py` — (a) customer_id values not matching the pattern C### and (b) order_date values later than today — and explain what results you get.
4. A 2-minute spoken explanation to your buddy of why raw data must not be edited.

15. Assessment

Rubric line	5 = excellent	3 = pass	0 = fail
Completeness of findings	All six defect types found with counts	At least five types	Fewer than four
Evidence	Every finding backed by query output or screenshot	Most backed	Statements without evidence
Business impact	Each defect tied to a wrong number or decision	Impact stated for most	None
Safe handling	Raw file untouched; Git history clean	Minor slip, corrected	Raw file edited
Documentation	README and dictionary allow a stranger to repeat the work	Understandable with gaps	Missing

Pass: every line 3 or more, average 3.5 or more, and Quiz Q1 (Annex C) at 70% or higher. Do not start Level 2 until A1 is passed.

Part 13 — Module Roadmap and Progress Tracker

Modules are issued one at a time. A module is released after the previous one is reviewed and approved. Status is updated at each release.

Module	Content	Status
1	Foundation: charter, organisation, onboarding, roadmap, 90-day plan, stack, lifecycle map, commercial model, sources, Assignment A1	**Issued (this document)**
2	Level 1 full lesson: data fundamentals, keys, quality, governance, KPIs, with quizzes	Next
3	Level 2 Excel, Project 1	Planned
4	Level 3 SQL on PostgreSQL, Project 2	Planned
5	Level 4 Python for analytics, Project 3	Planned
6	Level 5 Power BI and Tableau, Project 4	Planned
7	Level 6 data engineering, Projects 5–6	Planned
8	Level 7 predictive analytics, Project 7	Planned
9–12	Stages 01–17 in depth with templates; Project 8	Planned
13	Internal management portal (employees, training, clients, projects, tasks, inventories, QA, costing, billing, tickets, reports)	Planned
14	Sales kit: discovery questionnaires, proposals, demo scripts, SOW, pricing sheets, handover pack	Planned

Known limitations of this version

- Rate card and prices in Part 9 are illustrative only. The Founder must set VEL's real rates and verify vendor prices.
- Legal topics (data protection, contracts, tax) are described at awareness level; the company's lawyer and accountant must review the templates.
- Tool versions and menus change; each module lists the version it was written against.

Annex A — Data Audit Report Template (Stage 04 / Assignment A1)

Section	Content
1. Scope	Dataset name, source, date received, owner, row and column counts, file hash if available
2. Method	Tools and scripts used; confirmation that the raw file was not modified
3. Findings table	ID | Check | Count | Example rows | Business impact | Severity (High / Medium / Low)
4. Questions for the data owner	What does a blank region mean? Are negative quantities returns? Which total is the trusted one?
5. Recommendation	Fix at source / fix in clean layer / exclude / accept with note
6. Verdict	Fit for reporting: Yes / Yes after fixes / No
7. Sign-off	Author, reviewer, date

Annex B — Daily Log Template

```
Date:           Name:           Week/Day:
1. Done today:
2. Learned (one sentence per topic):
3. Blocked by / questions asked:
4. Commit(s) made (hash or message):
5. Plan for tomorrow:
Buddy initials:
```

Annex C — Quiz Q1 (Level 1, Week 1) and Trainer Answer Key

Trainer copy. Trainees receive the questions only. Pass mark 7 of 10.

#	Question	Answer key
1	Give one example each of structured, semi-structured and unstructured data.	Structured: sales table. Semi-structured: JSON from an API. Unstructured: customer email text, images.
2	What is a primary key, and why must it be unique and not blank?	Column(s) that identify one row; duplicates or blanks make rows ambiguous and break joins.
3	What is a foreign key? Give an example.	A column referencing another table's primary key, e.g. sales.customer_id to customers.customer_id.
4	Name four data quality dimensions and test one for sales_raw.csv.	Completeness, uniqueness, validity, consistency, accuracy, timeliness. E.g. completeness: 10 blank regions.
5	Why do we never edit raw data?	Traceability and repeatability; fixes happen in a clean layer so every number can be traced back.
6	Define KPI and give a good and a bad example.	A measurable value tied to a goal. Good: monthly revenue by region, defined precisely. Bad: "sales" with no definition or period.
7	A row has quantity -3. List two possible explanations and what to do.	Return/credit note or entry error. Do not guess; ask the owner and document the rule.

#	Question	Answer key
8	What is data governance in one sentence?	Agreed ownership, rules, quality standards and access controls for data.
9	Why is it unsafe to put a database password in a Git repository?	History keeps it forever and others can read it; use environment variables or a password manager.
10	Name the stages of the data lifecycle.	Collect, store, process/clean, analyse, share/report, retain/archive, delete.

Engineer must verify

This handbook and the training outputs support analytics work. Financial, medical, legal and other high-stakes conclusions drawn from data or models must be verified by a qualified professional before anyone acts on them.